Deep Learning Theory Notes

Neural Networks, CNN, RNN, LSTM, GAN & Transformers

---

1. What is Deep Learning?

> Deep Learning is a subset of Machine Learning that uses Artificial Neural Networks with multiple hidden layers to learn complex patterns from large amounts of data. It enables machines to automatically extract features without manual feature engineering.

The term 'deep' refers to the depth of the neural network — the number of layers stacked between the input and output. Deeper networks can learn more abstract and complex representations of data.

Key Characteristics

- Learns features automatically from raw data
- Requires large datasets for effective training
- Needs significant computational power (GPUs/TPUs)
- Outperforms traditional ML on unstructured data (images, audio, text)
- Inspired by the structure and function of the human brain

Real-World Applications

- Image and Face Recognition
- Speech Recognition and Synthesis
- Natural Language Processing (NLP)
- Autonomous Vehicles and Robotics
- Medical Imaging and Diagnosis
- Game Playing (AlphaGo, Chess AI)
- Fraud Detection and Cybersecurity

2. Artificial Neural Networks (ANN)

> An Artificial Neural Network (ANN) is the foundational building block of Deep Learning. It is a computational model inspired by biological neurons in the human brain, consisting of interconnected nodes (neurons) organized in layers.

Structure of an ANN

An ANN consists of three types of layers:

- Input Layer — Receives raw input data (features). Each neuron represents one feature.

- Hidden Layers — Intermediate layers that learn complex patterns through weighted transformations and activation functions. A network with multiple hidden layers is called a Deep Neural Network.
- Output Layer — Produces the final prediction. For regression, a single neuron outputs a continuous value; for classification, neurons output class probabilities.

How a Neuron Works

Each neuron receives inputs, multiplies them by learned weights, adds a bias term, and passes the result through an activation function to produce an output.

$$Output = Activation(\Sigma(weight_i \times input_i) + bias)$$

Activation Functions

Activation functions introduce non-linearity into the network, enabling it to learn complex patterns. Without them, a neural network would behave like a simple linear model.

- Sigmoid — Outputs values between 0 and 1. Used in binary classification output layers.
  - Formula: $\sigma(x) = 1 / (1 + e^{-x})$
  - Problem: Vanishing gradient for very large or small inputs
- ReLU (Rectified Linear Unit) — Outputs 0 for negative values, x for positive values. Most widely used in hidden layers.
  - Formula: $f(x) = \max(0, x)$
  - Advantage: Computationally efficient, reduces vanishing gradient problem
- Leaky ReLU — Allows a small negative slope to fix the dying ReLU problem.
  - Formula: $f(x) = x$ if $x > 0$, else αx (where α is a small constant like 0.01)
- Tanh (Hyperbolic Tangent) — Outputs values between -1 and 1. Zero-centered, better than sigmoid for hidden layers.
  - Formula: $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$
- Softmax — Converts outputs into a probability distribution. Used in multi-class classification output layers.
  - Formula: $\text{softmax}(x_i) = e^{x^i} / \Sigma e^{x^j}$

Weights and Biases

- Weights control the strength of connections between neurons
- Biases allow the model to shift the activation function
- Both weights and biases are learned during training through backpropagation
- Randomly initialized at the start of training

Loss Functions

Loss functions measure the difference between predicted and actual values. The goal of training is to minimize the loss.

- Mean Squared Error (MSE) — Used for regression tasks
- Binary Cross-Entropy — Used for binary classification
- Categorical Cross-Entropy — Used for multi-class classification
- Sparse Categorical Cross-Entropy — Used when labels are integers (not one-hot encoded)

Backpropagation

> Backpropagation is the algorithm used to train neural networks. It calculates the gradient of the loss function with respect to each weight, then uses gradient descent to update weights in the direction that reduces the loss.

1. Forward pass: Input flows through the network to produce a prediction
2. Loss is calculated by comparing prediction with actual output
3. Backward pass: Gradients are computed using the chain rule
4. Weights are updated: w = w - learning_rate × gradient
5. Process repeats for many epochs until the model converges

Optimizers

Optimizers are algorithms that update model weights during training to minimize the loss function.

- SGD (Stochastic Gradient Descent) — Updates weights using one sample at a time. Simple but noisy.
- Mini-Batch Gradient Descent — Updates weights using small batches of data. Balance between speed and accuracy.
- Adam (Adaptive Moment Estimation) — Most widely used optimizer. Adapts learning rate for each parameter using first and second moments of gradients.
- RMSprop — Adapts learning rate based on recent gradient magnitudes. Good for RNNs.
- Adagrad — Adapts learning rate per parameter. Good for sparse data.

3. Convolutional Neural Networks (CNN)

> A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed for processing structured grid data such as images. CNNs automatically learn spatial hierarchies of features through convolutional operations.

CNN Architecture

A typical CNN consists of the following layers in sequence:

- Convolutional Layer — Applies learnable filters (kernels) to the input to detect features like edges, textures, and shapes. Each filter produces a feature map.
- Activation Layer (ReLU) — Applies ReLU activation to introduce non-linearity after each convolution.

- Pooling Layer — Reduces spatial dimensions of feature maps to decrease computation and provide spatial invariance.
- Flattening Layer — Converts the 2D feature maps into a 1D vector for input to fully connected layers.
- Fully Connected Layer (Dense) — Learns the final classification or regression output from the flattened features.
- Output Layer — Produces final predictions using Softmax (classification) or linear activation (regression).

Convolution Operation

A filter (kernel) slides across the input image performing element-wise multiplication and summing the results to produce each value in the feature map. The filter learns to detect specific patterns.

$$\textit{Feature Map} = \textit{Input} \circledast \textit{Filter} + \textit{Bias}$$

- Filter Size: Typically 3×3 or 5×5
- Stride: Step size of the filter as it slides (controls output size)
- Padding: Adding zeros around the border to control output dimensions
- Multiple filters detect different features simultaneously

Types of Pooling

- Max Pooling — Takes the maximum value from each patch. Retains the most prominent features. Most commonly used.
- Average Pooling — Takes the average value from each patch. Provides smoother feature maps.
- Global Average Pooling — Averages the entire feature map to a single value. Used before output layers.

Popular CNN Architectures

Architecture	Year	Key Innovation	Layers
LeNet-5	1998	Pioneer CNN for digit recognition	7
AlexNet	2012	Deep CNN with ReLU and Dropout	8
VGGNet	2014	Very deep network with small 3×3 filters	16/19
GoogLeNet	2014	Inception modules for parallel convolutions	22
ResNet	2015	Residual skip connections for very deep networks	50/101/152
MobileNet	2017	Lightweight CNN for mobile devices	28

CNN Example (Keras)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

Applications of CNN

- Image Classification (ImageNet, CIFAR-10)
- Object Detection (YOLO, Faster R-CNN)
- Face Recognition
- Medical Image Analysis (X-ray, MRI)
- Self-Driving Car Perception
- Document and Handwriting Recognition

Advantages and Limitations

- Advantages: Automatically learns spatial features, parameter sharing reduces model size, translation invariant
- Limitations: Requires large labeled datasets, computationally expensive to train, not ideal for sequential data

4. Recurrent Neural Networks (RNN)

A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data by maintaining a hidden state that captures information from previous time steps. Unlike feedforward networks, RNNs have loops that allow information to persist.

How RNNs Work

At each time step t, the RNN takes the current input x_t and the previous hidden state h_{t-1} to compute a new hidden state h_t and an output y_t.

$$h_t = tanh(W_h \times h_{t-1} + W_x \times x_t + b)$$

- The same weights are shared across all time steps (weight sharing)
- The hidden state acts as the network's memory of previous inputs

- Final output depends on all previous inputs in the sequence
- Can handle variable-length input and output sequences

Types of RNN Architectures

- One-to-One — Single input, single output. Standard feedforward network.
- One-to-Many — Single input, sequence output. Example: Image captioning.
- Many-to-One — Sequence input, single output. Example: Sentiment analysis.
- Many-to-Many (same length) — Sequence to sequence of same length. Example: Video classification per frame.
- Many-to-Many (different length) — Encoder-Decoder structure. Example: Machine translation.

Vanishing Gradient Problem

> The vanishing gradient problem occurs during backpropagation through time (BPTT) when gradients become exponentially small as they are propagated back through many time steps. This makes it very difficult for standard RNNs to learn long-range dependencies.

- Gradients shrink exponentially with each time step during backpropagation
- The network effectively forgets information from the distant past
- Learning long-term dependencies becomes impossible
- Solution: Use LSTM or GRU instead of simple RNNs

Applications of RNN

- Time Series Prediction (stock prices, weather)
- Text Generation
- Speech Recognition
- Language Modelling
- Music Generation
- Video Analysis

RNN Example (Keras)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(64, activation='tanh', input_shape=(timesteps, features)),
    Dense(1, activation='sigmoid')
])
```

5. Long Short-Term Memory (LSTM)

> LSTM is an advanced type of RNN designed to solve the vanishing gradient problem. It uses a gating mechanism to selectively remember or forget information over long sequences, allowing it to capture long-range dependencies effectively.

LSTM Cell Structure

Each LSTM cell has two states and three gates that control information flow:

- Cell State (C_t) — The long-term memory of the network. Flows through the cell with minimal modification.
- Hidden State (h_t) — The short-term memory / working memory passed to the next time step and used as output.

The Three Gates

- Forget Gate — Decides what information from the previous cell state to throw away.
  - $f_t = \sigma(Wf \times [h_{t-1}, x_t] + bf)$
  - Output is between 0 (forget completely) and 1 (keep completely)
- Input Gate — Decides what new information to store in the cell state.
  - $i_t = \sigma(W_i \times [h_{t-1}, x_t] + b_i)$ → controls which values to update
  - $\tilde{C}_t = \tanh(Wc \times [h_{t-1}, x_t] + bc)$ → creates candidate values
- Output Gate — Decides what part of the cell state to output as the hidden state.
  - $o_t = \sigma(Wo \times [h_{t-1}, x_t] + bo)$
  - $h_t = o_t \times \tanh(C_t)$

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

The cell state is updated by partially forgetting old information and adding new candidate information, controlled by the forget and input gates.

GRU — Gated Recurrent Unit

> GRU is a simplified version of LSTM with only two gates (Update Gate and Reset Gate) instead of three. It has fewer parameters, trains faster, and often performs comparably to LSTM on many tasks.

- Update Gate — Combines forget and input gates into one
- Reset Gate — Controls how much past information to ignore
- No separate cell state — uses only hidden state
- Fewer parameters = faster training and less data required

LSTM Example (Keras)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(timesteps, features)),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
```

Applications of LSTM

- Sentiment Analysis
- Speech Recognition and Synthesis
- Stock Price Prediction
- Machine Translation
- Text Generation and Autocomplete
- Anomaly Detection in Time Series
- Video Captioning

RNN vs LSTM vs GRU Comparison

Feature	RNN	LSTM	GRU
Gates	None	3 (Forget, Input, Output)	2 (Update, Reset)
Memory	Short-term only	Long and short-term	Long and short-term
Vanishing Gradient	Severe problem	Solved	Solved
Speed	Fastest	Slower (more params)	Faster than LSTM
Parameters	Fewest	Most	Moderate
Best For	Short sequences	Long sequences	Moderate sequences

6. Generative Adversarial Networks (GAN)

A Generative Adversarial Network (GAN) is a deep learning architecture consisting of two neural networks — a Generator and a Discriminator — that compete against each other in a zero-sum game to produce realistic synthetic data.

GAN Architecture

- Generator (G) — Takes random noise as input and generates synthetic data samples (images, audio, text). Its goal is to fool the Discriminator into thinking the generated data is real.
- Discriminator (D) — Takes both real and generated samples as input and outputs the probability that a sample is real. Its goal is to correctly distinguish real from fake data.

The two networks are trained simultaneously in an adversarial process:

6. Generator tries to produce more realistic samples to fool the Discriminator
7. Discriminator tries to become better at detecting fake samples
8. As training progresses, the Generator improves until its outputs are indistinguishable from real data

Training Objective

The GAN training is a minimax game where the Generator minimizes and the Discriminator maximizes the following objective:

$$\min_G \max_D \; E[\log D(x)] + E[\log(1 - D(G(z)))]$$

- D(x) = probability that x is real (Discriminator output for real data)
- G(z) = generated fake sample from noise z
- D(G(z)) = probability that the fake sample is real

Common GAN Variants

- DCGAN (Deep Convolutional GAN) — Uses CNNs in both Generator and Discriminator for stable image generation.
- cGAN (Conditional GAN) — Conditions generation on class labels to control what is generated.
- CycleGAN — Translates images from one domain to another without paired training data (e.g., horses ↔ zebras).
- StyleGAN — Generates ultra-realistic human faces with fine style control at different levels.
- Pix2Pix — Paired image-to-image translation (e.g., sketch to photo).
- WGAN (Wasserstein GAN) — Improves training stability using Wasserstein distance instead of JS divergence.

GAN Training Challenges

- Mode Collapse — Generator produces limited variety, repeatedly generating the same outputs
- Training Instability — Difficult to balance Generator and Discriminator training
- Vanishing Gradients — When Discriminator is too strong, Generator gradients vanish

- Evaluation Difficulty — No clear objective metric for generation quality

Applications of GAN

- Photorealistic Image Generation (synthetic faces, scenes)
- Image-to-Image Translation
- Data Augmentation for limited datasets
- Super Resolution (enhancing image quality)
- Video Generation and Prediction
- Drug Discovery (molecular generation)
- Art and Music Generation
- Deepfake Generation and Detection

7. Transformers

> The Transformer is a deep learning architecture introduced in the paper 'Attention Is All You Need' (Vaswani et al., 2017). It relies entirely on a self-attention mechanism to process sequential data in parallel, without using recurrence or convolution.

Why Transformers?

- RNNs process sequences step-by-step and struggle with very long sequences
- Transformers process all positions in a sequence simultaneously (parallelization)
- Self-attention allows every token to directly attend to every other token
- Dramatically faster to train than RNNs on modern hardware (GPUs/TPUs)
- Scale extremely well with more data and parameters

Transformer Architecture

The original Transformer follows an Encoder-Decoder structure:

- Encoder — Processes the input sequence and builds contextual representations. Stacked N times.
- Decoder — Generates the output sequence using encoder representations and previously generated tokens. Stacked N times.

Each Encoder and Decoder block contains:

- Multi-Head Self-Attention — Computes attention weights for all token pairs simultaneously
- Feed-Forward Network — Applies two linear transformations with ReLU activation
- Layer Normalization — Normalizes activations for training stability
- Residual Connections — Skip connections around each sub-layer

Self-Attention Mechanism

> Self-attention allows the model to weigh the importance of different tokens in the sequence when encoding each token's representation. A token can attend to all other tokens — including itself — and the attention weights indicate how much focus to place on each.

Self-attention uses three vectors for each token:

- Query (Q) — What is this token looking for?
- Key (K) — What does this token offer / contain?
- Value (V) — What information does this token carry?

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k}) \times V$$

- QK^T computes compatibility (dot product) between queries and all keys
- Dividing by $\sqrt{d_k}$ prevents vanishing gradients for large dimensions
- Softmax normalizes the scores into attention weights (sum to 1)
- The output is a weighted sum of Values, guided by attention weights

Multi-Head Attention

Instead of computing a single attention function, Multi-Head Attention runs h parallel attention heads, each learning different types of relationships between tokens, then concatenates and projects their outputs.

$$MultiHead(Q,K,V) = Concat(head_1,...,head_h) \times W^O$$

- Each head learns to attend to different positional and semantic patterns
- Allows the model to jointly attend to information from different representation subspaces
- Typical models use 8 or 16 attention heads

Positional Encoding

Since Transformers have no inherent notion of order (unlike RNNs), positional encodings are added to input embeddings to inject information about the position of each token in the sequence.

$$PE(pos, 2i) = sin(pos / 10000^{(2i/dmodel)})$$

$$PE(pos, 2i+1) = cos(pos / 10000^{(2i/dmodel)})$$

Important Transformer Models

Model	Year	Type	Key Use
BERT	2018	Encoder only	Text classification, NER, Q&A
GPT	2018	Decoder only	Text generation, completion
GPT-2	2019	Decoder only	Large-scale text generation

GPT-3/4	2020/23	Decoder only	Few-shot learning, chatbots
T5	2019	Encoder-Decoder	Text-to-text tasks (translation, summarization)
ViT	2020	Encoder only	Image classification with patches
DALL-E	2021	Encoder-Decoder	Text-to-image generation

Applications of Transformers

- Machine Translation (Google Translate)
- Text Summarization
- Question Answering
- Chatbots and Conversational AI (ChatGPT, Claude)
- Code Generation (GitHub Copilot)
- Image Generation (DALL-E, Stable Diffusion)
- Protein Structure Prediction (AlphaFold)
- Document Understanding

8. Regularization Techniques in Deep Learning

Regularization techniques help prevent overfitting and improve generalization of deep learning models to unseen data.

Dropout

> Dropout randomly deactivates a fraction of neurons during each training step. This prevents neurons from co-adapting and forces the network to learn more robust, distributed representations.

- During training: Each neuron is dropped with probability p (typically 0.2–0.5)
- During inference: All neurons are active, weights are scaled by (1-p)
- Acts as an ensemble of many different network architectures

```
model.add(Dropout(0.5))  # Drops 50% of neurons
```

Batch Normalization

> Batch Normalization normalizes the input to each layer across the mini-batch during training. It stabilizes and accelerates training by reducing internal covariate shift.

- Normalizes activations to have mean ≈ 0 and variance ≈ 1
- Allows higher learning rates and reduces sensitivity to initialization
- Provides mild regularization effect, sometimes reducing need for Dropout
- Applied before or after the activation function

L1 and L2 Regularization

- L1 Regularization (Lasso) — Adds the sum of absolute values of weights to the loss. Promotes sparsity (many weights become 0).
- L2 Regularization (Ridge / Weight Decay) — Adds the sum of squared weights to the loss. Keeps weights small but rarely zero. Most commonly used in deep learning.

$$L2\ Loss = Original\ Loss + \lambda \times \Sigma(w_i^2)$$

Early Stopping

Training is stopped when the validation loss stops improving for a specified number of epochs (patience), preventing the model from overfitting to training data.

Data Augmentation

Artificially increases the size and diversity of the training dataset by applying random transformations to existing samples.

- Image: Random flipping, rotation, cropping, color jittering, noise
- Text: Synonym replacement, back-translation, random deletion
- Audio: Speed perturbation, pitch shift, noise addition

9. Transfer Learning

> Transfer Learning is a technique where a model pre-trained on a large dataset is reused as the starting point for a new, related task. Instead of training from scratch, the pre-trained model's learned features are transferred and fine-tuned on the target dataset.

Why Transfer Learning?

- Training deep models from scratch requires millions of labeled samples
- Pre-trained models already understand low-level features (edges, textures in images; grammar in text)
- Significantly reduces training time and computational cost
- Achieves high accuracy even with small target datasets
- Especially powerful when source and target tasks are related

Strategies

- Feature Extraction — Freeze all pre-trained layers. Add and train only new output layers. Best when datasets are small and similar.
- Fine-Tuning — Unfreeze some or all pre-trained layers and retrain with a very low learning rate. Best when target dataset is moderately large.

- Full Retraining — Train all layers on the new dataset. Only feasible when the target dataset is very large.

Transfer Learning Example (Keras)

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224,224,3))
base_model.trainable = False  # Freeze pretrained layers

x = GlobalAveragePooling2D()(base_model.output)
x = Dense(256, activation='relu')(x)
output = Dense(10, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
```

Popular Pre-Trained Models

Model	Domain	Pre-trained On	Common Use
VGG16/19	Vision	ImageNet (1.2M images)	Image classification, feature extraction
ResNet50	Vision	ImageNet	Image classification, object detection
InceptionV3	Vision	ImageNet	Efficient image classification
EfficientNet	Vision	ImageNet	High accuracy with fewer params
BERT	NLP	Books + Wikipedia	Text classification, NER, Q&A
GPT-2/3	NLP	Web text (40GB+)	Text generation, summarization
ViT	Vision	ImageNet-21k	Image classification with patches

10. Deep Learning Model Development Workflow

Building an effective deep learning model follows a systematic workflow:

1. Define the Problem — Clarify task type (classification, regression, generation), output format, and success criteria
2. Collect and Prepare Data — Gather sufficient labeled data; apply cleaning, augmentation, and preprocessing
3. Choose Architecture — Select CNN for images, LSTM/Transformer for sequences, GAN for generation

4. Design the Model — Define layers, activations, loss function, and optimizer
5. Train the Model — Run forward and backward passes over epochs using batches of data
6. Monitor Training — Track training and validation loss/accuracy; use callbacks (early stopping, LR scheduler)
7. Evaluate the Model — Test on held-out data; compute metrics (accuracy, F1, MAE, etc.)
8. Tune Hyperparameters — Adjust learning rate, batch size, architecture, regularization
9. Deploy the Model — Export as TensorFlow SavedModel, ONNX, or TorchScript for production use

11. Deep Learning Frameworks and Libraries

TensorFlow / Keras

Developed by Google. Keras provides a high-level API on top of TensorFlow for fast model building. TensorFlow handles large-scale deployment with TensorFlow Serving, TFLite, and TF.js.

- Sequential and Functional API for model building
- Pre-built layers for CNN, RNN, LSTM, Transformer components
- TensorBoard for training visualization
- Supports GPU/TPU acceleration

PyTorch

Developed by Meta AI. PyTorch is widely used in research due to its dynamic computation graph (define-by-run). It is highly Pythonic and flexible.

- Dynamic computation graph for flexible model design
- torch.nn module for layer building
- Automatic differentiation with autograd
- TorchScript for production deployment
- Most popular framework in academic research

Hugging Face Transformers

A library providing thousands of pre-trained Transformer models for NLP, vision, and audio tasks. Simplifies transfer learning with simple APIs.

```
from transformers import BertForSequenceClassification, Trainer
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
```

12. Deep Learning Architectures — Master Comparison

Architecture	Input Type	Key Mechanism	Primary Applications
ANN	Tabular/Flat	Fully connected layers	Classification, regression on structured data
CNN	Images/Grids	Convolution + Pooling	Image classification, object detection, segmentation
RNN	Sequences	Recurrent hidden state	Short text, time series
LSTM	Long Sequences	Gated cell state	NLP, speech, long time series
GRU	Long Sequences	Simplified gating	Faster alternative to LSTM
GAN	Random Noise	Generator vs Discriminator	Image synthesis, data augmentation
Transformer	Sequences/Patches	Self-attention	NLP, multimodal AI, large language models

---

Conclusion

Deep Learning has revolutionized Artificial Intelligence by enabling machines to learn complex patterns directly from raw data. Artificial Neural Networks form the foundation, while specialized architectures such as CNNs excel at spatial data, RNNs and LSTMs handle sequential and temporal data, GANs generate realistic synthetic data, and Transformers have redefined the state-of-the-art in Natural Language Processing and multimodal AI. Mastering regularization, transfer learning, and the deep learning development workflow is essential for building robust, production-ready AI systems.